TP n°2 – Requêtes, dépendances fonctionnelles et normalisation

Base de données : Gestion des cours universitaires

Oracle APEX | SQL Workshop

Nom	N'GUESSAN
Prénom	Donogbo Jean-Aristide
Numéro étudiant	12014142
Spécialité	INFO2
Cours	Base de données
Enseignant	Youcef SAMIR

En cas de difficulté à se connecter, se rapporter à la section « Préparation de l'environnement de travail » de la série de TP n°1. Les requêtes s'exécutent dans **Oracle APEX > SQL Workshop > SQL Commands**.

Ce rapport a été réalisé avec l'utilisation du langage LaTex qui est très connu pour écrire des rapports.

Exercice n°1 – Requêtes SQL avancées

Q1 – Département avec le budget le plus élevé

```sql
SELECT dept_name
FROM department
WHERE budget = (SELECT MAX(budget) FROM department);
```

Q2 – Enseignants gagnant plus que le salaire moyen

```sql
SELECT name, salary
FROM teacher
WHERE salary > (SELECT AVG(salary) FROM teacher)
ORDER BY salary DESC;
```

Q3 – Étudiants ayant suivi plus de 2 cours d'un même enseignant (avec `HAVING`)

```sql
SELECT te.ID AS id_enseignant, t.name AS enseignant,
       tk.ID AS id_etudiant,   s.name AS etudiant,
       COUNT(*) AS nb_cours
FROM teaches te
JOIN takes tk ON te.course_id = tk.course_id
             AND te.sec_id    = tk.sec_id
             AND te.semester  = tk.semester
             AND te.year      = tk.year
JOIN teacher t ON te.ID = t.ID
JOIN student s ON tk.ID = s.ID
GROUP BY te.ID, t.name, tk.ID, s.name
HAVING COUNT(*) > 2;
```

Q4 – Même résultat sans HAVING

```sql
SELECT *
FROM (
    SELECT te.ID AS id_enseignant, t.name AS enseignant,
           tk.ID AS id_etudiant,   s.name AS etudiant,
           COUNT(*) AS nb_cours
    FROM teaches te
    JOIN takes tk ON te.course_id = tk.course_id
                 AND te.sec_id    = tk.sec_id
                 AND te.semester  = tk.semester
                 AND te.year      = tk.year
    JOIN teacher t ON te.ID = t.ID
    JOIN student s ON tk.ID = s.ID
    GROUP BY te.ID, t.name, tk.ID, s.name
) sub
WHERE sub.nb_cours > 2;
```

Q5 – Étudiants n'ayant pas suivi de cours avant 2010

```sql
SELECT s.ID, s.name
FROM student s
WHERE s.ID NOT IN (
    SELECT ID FROM takes WHERE year < 2010
);
```

Q6 – Enseignants dont le nom commence par E

```sql
SELECT name
FROM teacher
WHERE name LIKE 'E%';
```

Q7 – Enseignants percevant le quatrième salaire le plus élevé

```sql
SELECT name, salary
FROM teacher
WHERE salary = (
    SELECT MIN(salary)
    FROM (
        SELECT DISTINCT salary
        FROM teacher
        ORDER BY salary DESC
        FETCH NEXT 4 ROWS ONLY
    )
);
```

Q8 – Les trois enseignants avec les salaires les moins élevés (ordre décroissant)

```sql
SELECT name, salary
FROM (
    SELECT name, salary
    FROM teacher
    ORDER BY salary ASC
    FETCH NEXT 3 ROWS ONLY
)
ORDER BY salary DESC;
```

Q9 – Étudiants ayant suivi un cours en automne 2009 (avec IN)

```
SELECT name
FROM student
WHERE ID IN (
    SELECT ID FROM takes
    WHERE semester = 'Fall' AND year = 2009
);
```

Q10 – Étudiants ayant suivi un cours en automne 2009 (avec SOME)

```
SELECT name
FROM student
WHERE ID = SOME (
    SELECT ID FROM takes
    WHERE semester = 'Fall' AND year = 2009
);
```

Q11 – Étudiants ayant suivi un cours en automne 2009 (NATURAL INNER JOIN)

```
SELECT DISTINCT s.name
FROM student s
NATURAL INNER JOIN takes tk
WHERE tk.semester = 'Fall' AND tk.year = 2009;
```

Q12 – Étudiants ayant suivi un cours en automne 2009 (avec EXISTS)

```
SELECT name
FROM student s
WHERE EXISTS (
    SELECT 1 FROM takes tk
    WHERE tk.ID = s.ID
      AND tk.semester = 'Fall'
      AND tk.year = 2009
);
```

Q13 – Toutes les paires d'étudiants ayant suivi au moins un cours ensemble

```
SELECT DISTINCT s1.name AS etudiant_1, s2.name AS etudiant_2
FROM takes t1
JOIN takes t2 ON t1.course_id = t2.course_id
             AND t1.sec_id    = t2.sec_id
             AND t1.semester  = t2.semester
             AND t1.year      = t2.year
             AND t1.ID < t2.ID
JOIN student s1 ON t1.ID = s1.ID
JOIN student s2 ON t2.ID = s2.ID;
```

Q14 – Pour chaque enseignant ayant assuré un cours : total d'étudiants (trié décroissant)

Si un étudiant a suivi deux cours différents du même enseignant, il est compté deux fois.

```sql
SELECT t.name AS enseignant, COUNT(*) AS nb_etudiants
FROM teacher t
JOIN teaches te ON t.ID = te.ID
JOIN takes tk   ON te.course_id = tk.course_id
               AND te.sec_id    = tk.sec_id
               AND te.semester  = tk.semester
               AND te.year      = tk.year
GROUP BY t.ID, t.name
ORDER BY nb_etudiants DESC;
```

Q15 – Pour chaque enseignant (même sans cours) : total d'étudiants (trié décroissant)

```sql
SELECT t.name AS enseignant, COUNT(tk.ID) AS nb_etudiants
FROM teacher t
LEFT JOIN teaches te ON t.ID = te.ID
LEFT JOIN takes tk   ON te.course_id = tk.course_id
                    AND te.sec_id    = tk.sec_id
                    AND te.semester  = tk.semester
                    AND te.year      = tk.year
GROUP BY t.ID, t.name
ORDER BY nb_etudiants DESC;
```

Q16 – Pour chaque enseignant : nombre total de grades A attribués

```sql
SELECT t.name AS enseignant, COUNT(*) AS nb_A
FROM teacher t
JOIN teaches te ON t.ID = te.ID
JOIN takes tk   ON te.course_id = tk.course_id
               AND te.sec_id    = tk.sec_id
               AND te.semester  = tk.semester
               AND te.year      = tk.year
WHERE tk.grade = 'A'
GROUP BY t.ID, t.name;
```

Q17 – Toutes les paires enseignant-étudiant avec le nombre de cours suivis ensemble

```sql
SELECT t.name AS enseignant, s.name AS etudiant,
       COUNT(*) AS nb_cours
FROM teacher t
JOIN teaches te ON t.ID = te.ID
JOIN takes tk   ON te.course_id = tk.course_id
               AND te.sec_id    = tk.sec_id
               AND te.semester  = tk.semester
               AND te.year      = tk.year
JOIN student s  ON tk.ID = s.ID
GROUP BY t.ID, t.name, s.ID, s.name;
```

Q18 – Paires enseignant-étudiant où l'étudiant a suivi au moins 2 cours de l'enseignant

```sql
SELECT t.name AS enseignant, s.name AS etudiant,
       COUNT(*) AS nb_cours
FROM teacher t
JOIN teaches te ON t.ID = te.ID
JOIN takes tk   ON te.course_id = tk.course_id
               AND te.sec_id    = tk.sec_id
               AND te.semester  = tk.semester
               AND te.year      = tk.year
JOIN student s  ON tk.ID = s.ID
GROUP BY t.ID, t.name, s.ID, s.name
HAVING COUNT(*) >= 2;
```

Exercice n°2 – Dépendances fonctionnelles et formes normales

Pour chaque schéma, on détermine la forme normale la plus avancée (1NF, 2NF, 3NF, BCNF) et on propose une décomposition si la relation n'est pas normalisée.

Q1 – $R(A, B, C)$, $F = \{A \longrightarrow B,\ B \longrightarrow C\}$

Clé candidate : $A^+ = \{A, B, C\} = R$, donc A est la seule clé candidate.

Analyse :

- $B \longrightarrow C$ est une dépendance transitive ($A \longrightarrow B \longrightarrow C$) : R n'est **pas en 3NF**.
- B n'est pas une superclé $\Rightarrow$ violation de BCNF également.

Décomposition en BCNF :

- $R_1(B, C)$ avec $\{B \longrightarrow C\}$ – B est clé $\Rightarrow$ BCNF.
- $R_2(A, B)$ avec $\{A \longrightarrow B\}$ – A est clé $\Rightarrow$ BCNF.

Q2 – $R(A, B, C)$, $F = \{A \longrightarrow C,\ A \longrightarrow B\}$

Clé candidate : $A^+ = \{A, B, C\} = R$, donc A est la seule clé candidate.

Analyse : Toutes les dépendances ont A (superclé) en partie gauche. La relation est en **BCNF** – forme normale la plus avancée, aucune décomposition nécessaire.

Q3 – $R(A, B, C)$, $F = \{AB \longrightarrow C,\ C \longrightarrow B\}$

Clés candidates :

- $(AB)^+ = \{A, B, C\} = R \Rightarrow AB$ est une superclé minimale.
- $(AC)^+ = \{A, C, B\} = R \Rightarrow AC$ est aussi une superclé minimale.
- Deux clés candidates : AB et AC.

Analyse :

- $C \longrightarrow B$: C n'est pas une superclé $\Rightarrow$ **violation BCNF**.
- B est un attribut premier (dans la clé AB), donc $C \longrightarrow B$ ne viole pas la 3NF.
- R est en **3NF** mais **pas en BCNF**.

Décomposition en BCNF :

- $R_1(B, C)$ avec $\{C \longrightarrow B\}$ – C est clé $\Rightarrow$ BCNF.

— $R_2(A, C)$ – AC est clé $\Rightarrow$ BCNF.

Attention : Cette décomposition est sans perte d'information (jointure sur C) mais elle ne préserve pas la dépendance $AB \longrightarrow C$.

Exercice n°3 – Fermetures, super-clés et BCNF

Q1 – $R(A, B, C, D, E)$, $F = \{A \longrightarrow BC,\ CD \longrightarrow E,\ B \longrightarrow D,\ E \longrightarrow A\}$

Déduction de 16 dépendances par les axiomes d'Armstrong :

1. $A \longrightarrow B$ (décomposition de $A \longrightarrow BC$)
2. $A \longrightarrow C$ (décomposition de $A \longrightarrow BC$)
3. $A \longrightarrow D$ (transitivité : $A \longrightarrow B$, $B \longrightarrow D$)
4. $A \longrightarrow BC$ (donné)
5. $A \longrightarrow BD$ (union : $A \longrightarrow B$, $A \longrightarrow D$)
6. $A \longrightarrow CD$ (union : $A \longrightarrow C$, $A \longrightarrow D$)
7. $A \longrightarrow BCD$ (union de 1, 2, 3)
8. $A \longrightarrow E$ (transitivité : $A \longrightarrow CD$, $CD \longrightarrow E$)
9. $A \longrightarrow BCDE$ (union de 7 et 8)
10. $E \longrightarrow A$ (donné)
11. $E \longrightarrow B$ (transitivité : $E \longrightarrow A$, $A \longrightarrow B$)
12. $E \longrightarrow C$ (transitivité : $E \longrightarrow A$, $A \longrightarrow C$)
13. $E \longrightarrow D$ (transitivité : $E \longrightarrow A$, $A \longrightarrow D$)
14. $E \longrightarrow BC$ (union de 11 et 12)
15. $E \longrightarrow ABCD$ (transitivité : $E \longrightarrow A$, $A \longrightarrow BCD$)
16. $CD \longrightarrow A$ (transitivité : $CD \longrightarrow E$, $E \longrightarrow A$)

Q2 – $R(A, B, C, D, E, F)$, $F = \{A \longrightarrow BCD,\ BC \longrightarrow DE,\ B \longrightarrow D,\ D \longrightarrow A\}$

(a) Fermeture de B et de $\{A, B\}$

Fermeture de B :

$$B \xrightarrow{B \longrightarrow D} BD \xrightarrow{D \longrightarrow A} ABD \xrightarrow{A \longrightarrow BCD} ABCD \xrightarrow{BC \longrightarrow DE} ABCDE$$

$$\boxed{B^+ = \{A, B, C, D, E\}}$$

$F \notin B^+$, donc B n'est **pas** une superclé de $R(A, B, C, D, E, F)$.

Fermeture de $\{A, B\}$:

$$AB \xrightarrow{A \longrightarrow BCD} ABCD \xrightarrow{BC \longrightarrow DE} ABCDE$$

$$\boxed{\{A, B\}^+ = \{A, B, C, D, E\}}$$

$F \notin \{A, B\}^+$, donc $\{A, B\}$ n'est **pas** une superclé de R.

(b) $\{A, F\}$ est une superclé

$$AF \xrightarrow{A \longrightarrow BCD} ABCDF \xrightarrow{BC \longrightarrow DE} ABCDEF$$

$$\{A, F\}^+ = \{A, B, C, D, E, F\} = R$$

Donc $\{A, F\}$ est bien une **superclé** de R. $\square$

(c) R est-elle en BCNF ?

Pour être en BCNF, toute partie gauche non triviale doit être une superclé de R.

- $A \longrightarrow BCD$: $A^+ = \{A, B, C, D, E\} \neq R \Rightarrow$ **violation BCNF**.
- $BC \longrightarrow DE$: $(BC)^+ \neq R \Rightarrow$ violation BCNF.
- $B \longrightarrow D$: $B^+ \neq R \Rightarrow$ violation BCNF.
- $D \longrightarrow A$: $D^+ \neq R \Rightarrow$ violation BCNF.

R **n'est pas en BCNF**. Décomposition (violation choisie : $A \longrightarrow BCD$) :

- $R_1(A, B, C, D)$ avec $F_1 = \{A \longrightarrow BCD,\ BC \longrightarrow D,\ B \longrightarrow D,\ D \longrightarrow A\}$.
- $R_2(A, E, F)$ sans dépendance non triviale $\Rightarrow$ déjà en BCNF.

On décompose ensuite R_1 (violation $B \longrightarrow D$) :

- $R_{1a}(B, D)$ avec $\{B \longrightarrow D\}$ – B est clé $\Rightarrow$ BCNF.
- $R_{1b}(A, B, C)$ avec $\{A \longrightarrow BC,\ BC \longrightarrow A\}$ – A et BC sont clés $\Rightarrow$ BCNF.

Décomposition finale : $R_{1a}(B, D),\ R_{1b}(A, B, C),\ R_2(A, E, F)$.

Q3 – Décompositions sans perte et avec perte

(a) $R_1(A, B, C)$ et $R_2(A, D, E)$ – sans perte d'information

$R_1 \cap R_2 = \{A\}$. Comme $A \longrightarrow BCD \in F^+$, l'attribut A est une superclé de $R_1(A, B, C)$. La condition de Heath est satisfaite :

$$R_1 \bowtie R_2 = R \qquad \square$$

(b) $R_1(A, B, C)$ et $R_2(C, D, E)$ – avec perte d'information

$R_1 \cap R_2 = \{C\}$. Or C ne détermine ni A, ni B, ni D, ni E dans F^+ : C n'est superclé ni de R_1 ni de R_2.

La jointure naturelle peut produire des n-uplets parasites (*spurious tuples*) :

$$R_1 \bowtie R_2 \supseteq R \quad \text{(inclusion stricte possible)}$$

La décomposition est donc **avec perte d'information**. $\square$

Exercice n°4 – Implémentation Python

Les structures de données utilisées suivent les conventions du sujet : une relation est un `set` de chaînes, et les dépendances sont une liste de paires de deux `set`.

Q1 – Afficher les dépendances fonctionnelles

```python
def printDependencies(F: "list of dependencies"):
    for alpha, beta in F:
        print("\t", alpha, " --> ", beta)
```

Q2 – Afficher les relations

```python
def printRelations(T: "list of relations"):
    for R in T:
        print("\t", R)
```

Q3 – Calculer le powerset

```python
import itertools

def powerSet(inputset: "set"):
    _result = []
    for r in range(1, len(inputset) + 1):
        _result += map(set, itertools.combinations(inputset, r))
    return _result
```

Q4 – Fermeture d'un ensemble d'attributs K

```python
def closure(F: "list of dependencies", K: "set") -> set:
    """Retourne la fermeture de K sous F."""
    clot = set(K)
    changed = True
    while changed:
        changed = False
        for alpha, beta in F:
            if alpha.issubset(clot) and not beta.issubset(clot):
                clot |= beta
                changed = True
    return clot
```

Q5 – Fermeture de F

```python
def closureF(F: "list of dependencies", R: "set") -> list:
    """Retourne F+ : toutes les DF deductibles de F."""
    result = []
    for alpha in powerSet(R):
        alpha_set = set(alpha)
        cl = closure(F, alpha_set)
        for beta in powerSet(cl):
            beta_set = set(beta)
            if not beta_set.issubset(alpha_set):
                result.append([alpha_set, beta_set])
    return result
```

Q6 – α détermine-t-il fonctionnellement β ?

```python
def determines(F: "list of dependencies",
               alpha: "set", beta: "set") -> bool:
    """Retourne vrai si alpha --> beta est dans F+."""
    return beta.issubset(closure(F, alpha))
```

Q7 – K est-il une super-clé ?

```python
def isSuperKey(F: "list of dependencies",
               R: "set", K: "set") -> bool:
    """Retourne vrai si K est une super-cle de R."""
    return closure(F, K) == R
```

Q8 – K est-il une clé candidate ?

```python
def isCandidateKey(F: "list of dependencies",
                   R: "set", K: "set") -> bool:
    """Retourne vrai si K est une cle candidate (supercl  minimale)."""
    if not isSuperKey(F, R, K):
        return False
    # Verifier qu'aucun sous-ensemble strict n'est deja une supercl
    for attr in K:
        subset = K - {attr}
        if subset and isSuperKey(F, R, subset):
            return False
    return True
```

Q9 – Liste de toutes les clés candidates

```python
def allCandidateKeys(F: "list of dependencies", R: "set") -> list:
    """Retourne la liste de toutes les cles candidates de R."""
    return [set(K) for K in powerSet(R)
            if isCandidateKey(F, R, set(K))]
```

Q10 – Liste de toutes les super-clés

```python
def allSuperKeys(F: "list of dependencies", R: "set") -> list:
    """Retourne la liste de toutes les super-cles de R."""
    return [set(K) for K in powerSet(R)
            if isSuperKey(F, R, set(K))]
```

Q11 – Trouver une clé candidate

```python
def oneCandidateKey(F: "list of dependencies", R: "set") -> set:
    """Retourne une cle candidate de R."""
    key = set(R)
    for attr in list(R):
        candidate = key - {attr}
        if candidate and isSuperKey(F, R, candidate):
            key = candidate
    return key
```

Q12 – R est-elle en BCNF ?

```python
def isBCNF(F: "list of dependencies", R: "set") -> bool:
    """Retourne vrai si R est en BCNF sous F."""
    for alpha, beta in F:
        alpha_set = set(alpha)
        beta_set  = set(beta)
        # Ignorer les DF triviales
        if beta_set.issubset(alpha_set):
            continue
        if not isSuperKey(F, R, alpha_set):
            return False
    return True
```

Q13 – Le schéma est-il en BCNF ?

```python
def isSchemaBCNF(F: "list of dependencies",
                 T: "list of relations") -> bool:
    """Retourne vrai si toutes les relations de T sont en BCNF."""
    for R in T:
        # Projeter F sur R
        F_R = [[set(a), set(b)] for a, b in F
               if set(a).issubset(R) and set(b).issubset(R)]
        if not isBCNF(F_R, R):
            return False
    return True
```

Q14 – Algorithme de décomposition en BCNF

```python
def bcnfDecomposition(F: "list of dependencies",
                      T: "list of relations") -> list:
    """
    Decompose chaque relation de T en BCNF.
    Retourne la liste finale de relations toutes en BCNF.
    """
    result   = []
    worklist = list(T)

    while worklist:
        R   = worklist.pop()
        F_R = [[set(a), set(b)] for a, b in F
               if set(a).issubset(R) and set(b).issubset(R)]

        violation = None
        for alpha, beta in F_R:
            if not beta.issubset(alpha) and not isSuperKey(F_R, R,
                alpha):
                violation = (alpha, beta)
                break

        if violation is None:
            result.append(R)          # R est en BCNF
        else:
            alpha, beta = violation
            R1 = alpha | beta         # R1 = alpha U beta
            R2 = R - (beta - alpha)   # R2 = R - (beta \ alpha)
            worklist.append(R1)
            worklist.append(R2)

    return result
```